Licence Informatique
Parcours « Méthodes Informatiques Appliquées à la Gestion d'Entreprises »

Free Fight
Rapport de projet

Année universitaire 2025 - 2026

Farah Bentekhici
Amine Chboukhat
Luc Levêque

Programmation Concurrentielle et Interface Interactive
Encadré par M. Nicolas Sabouret

Table des matières

1. Introduction **3**
1.1. Cahier des charges 3
1.2. Interface attendue 3
2. Analyse globale **4**
2.1. Gestion des bâtiments du village 4
2.2. Gestion des troupes du joueur 4
2.3. Gestion spécifique du Pekka 4
2.4. Défense du château de clan 4
2.5. Gestion du combat 4
2.6. Gestion des étoiles (Score) et de l'or 4
2.7. Amélioration des troupes 5
2.8. Affichage graphique 5
2.9. Arrêt de la bataille (Abandonner / 100% / Mort des troupes) 5
2.10. Gestion du déroulement d'une partie 5
2.11. Gestion des animations 5
2.12. Interface utilisateur 5
3. Plan de développement **6**
4. Conception générale **7**
4.1. Structure du jeu 7
4.2. Blocs fonctionnels et communication 7
4.3. Explication de l'architecture 8
4.4. Threads principaux 8
5. Conception détaillée **9**
5.1. Création des troupes du joueur 9
5.2. Affichage graphique 10
5.3. Gestion des troupes du joueur 11
5.4. Gestion du temps de jeu 12
5.5. Gestion spécifique du Pekka 14
5.6. Gestion du combat 15
5.7. Gestion des étoiles et de la fin de partie 17
5.8. Gestion des bâtiments et des défenses 19
5.9. Défense du château de clan 21
5.10. Gestion des niveaux 22
5.11 Amélioration des troupes 25
6. Résultat **28**
7. Documentation utilisateur **31**
7.1 Pré Requis logiciels 31
7.2 Lancement du jeu 31
7.3 Démarrer une partie 31
7.4. Commandes utilisateur 31
7.5. Objectif du jeu 31
7.6. Fin de partie 31
8. Documentation développeur **33**
8.1 Point d'entrée du programme 33
8.2. Classes principales à connaître 33
8.3. Constantes importantes à modifier 34
8.4. Fonctionnalités pouvant être ajoutées 35
9. Conclusion et perspectives **36**
9.1. Retour sur les difficultés rencontrées 36

1. Introduction

Dans le cadre du cours de programmation concurrentielle, il nous a été demandé de réaliser un jeu en Java utilisant des *threads* et avec un minimum d'interfaces graphiques. Nous avons choisi de faire une version simplifiée de *ClashOfClans* qu'on nommera *FreeFight*.

1.1. Cahier des charges

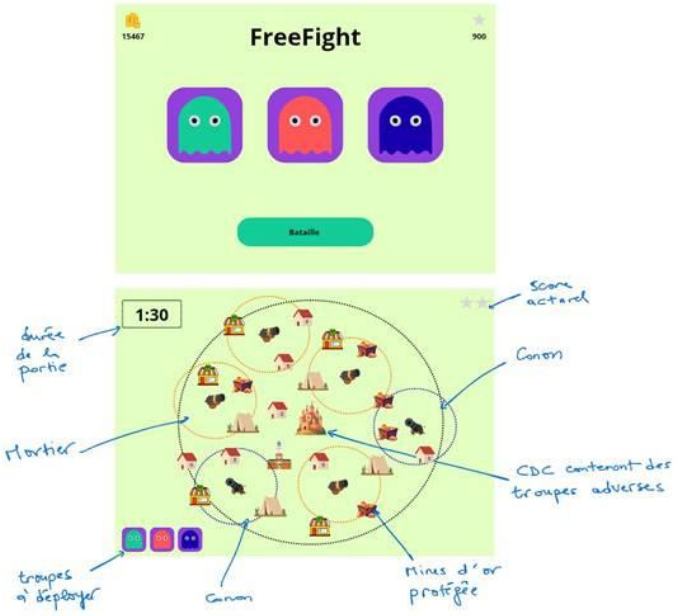
Le principe du jeu repose sur une battle entre des villageois, le joueur devra attaquer un village et essayer de le piller au maximum et gratter en même temps de l'or pour améliorer ses troupes. Il dispose de trois types de guerriers (Sorciers, barbares et les pekka) qu'il disposera comme il souhaite autour du village pour le détruire au maximum.

Les Pekka auront la possibilité d'être dirigés et cracher du feu sous la commande du joueur.

Une destruction totale du village vaudra 3 étoiles, la moitié 2 étoiles et la destruction de l'hôtel de ville 1 étoile. Quelques bâtiments du village peuvent se défendre (Canon, mortier) et un nombre fixe de troupes adverses sortent du château de clan pour défendre le village. Un effet visuel est déclenché à chaque fois qu'on prend une étoile ainsi qu'à la fin de la partie avec le score finale et la quantité d'or obtenue.

1.2. Interface attendue

Un dessin explicatif du rendu souhaité fait sur Canva :



2. Analyse globale

Dans cette rubrique, on aborde les fonctionnalités principales constituant le jeu ainsi que leurs sous-fonctionnalités. Le niveau de difficulté de la réalisation de la tâche ainsi que sa priorité seront également mentionnés.

2.1. Gestion des bâtiments du village

Cette tâche correspond donc à mettre en place les bâtiments qui composent le village que les troupes ennemies prendront pour cible.

- Priorité : Haute
- Difficulté : Moyenne

2.2. Gestion des troupes du joueur

Cette tâche correspond à la création des troupes du joueur destinés à attaquer le village ennemi. Il y a trois troupes au total : Sorciers, barbares et les pekka.

- Priorité : Haute.
- Difficulté : Basse.

2.3. Gestion spécifique du Pekka

Le Pekka sera manipulé par le joueur qui décidera où aller et pourra cracher du feu à sa demande.

- Priorité : basse.
- Difficulté : haute.

2.4. Défense du château de clan

La défense du château de clan est composée de troupes ennemies qui sortiront du château pour défendre le village quand les troupes du joueur entreront dans un certain périmètre.

- Priorité : Haute
- Difficulté : Moyenne

2.5. Gestion du combat

Le combat sera représenté par l'affrontement entre les troupes de l'attaquant et les troupes qui défendent le village et qui sortiront du château de clan.

- Priorité : Haute
- Difficulté : Basse

2.6. Gestion des étoiles (Score) et de l'or

La gestion des étoiles se fera au score réalisé par l'attaquant. Le village devra être détruit à 50% pour obtenir une étoile. La destruction du bâtiment principal (hôtel de ville) permet d'obtenir une autre étoile. Et les trois étoiles seront obtenues uniquement si tout le village est détruit.

- Priorité : Haute
- Difficulté : Basse

2.7. Amélioration des troupes

Les troupes peuvent être améliorées en échange d'une certaine somme d'or pour pouvoir être plus résistants et faire plus de dégâts.

- Priorité : Basse
- Difficulté : Basse

2.8. Affichage graphique

L'affichage graphique consiste à faire afficher le village composé de ses bâtiments et de ses défenses sur l'écran ainsi que les troupes et les différentes animations.

- Priorité : Haute
- Difficulté : Moyenne

2.9. Arrêt de la bataille (Abandonner / 100% / Mort des troupes)

La bataille s'arrête uniquement dans trois cas. Soit l'attaquant abandonne en cliquant sur un bouton, soit le village est entièrement détruit, soit les défenses ont pu tuer l'entièreté des troupes ennemies.

- Priorité : Basse
- Difficulté : Basse

2.10. Gestion du déroulement d'une partie

La partie commence avec l'utilisateur qui déploie ses troupes. Les troupes attaquent le village et détruisent les bâtiments au fur et à mesure. Les défenses défendent le village en tentant de tuer les troupes ennemies jusqu'au dernier. Au fur et à mesure de la partie, le joueur pourra déplacer le PEKKA et lui faire cracher du feu sur les bâtiments. La partie se termine lorsqu'une des conditions citées dans Arrêt de la bataille est respectée.

- Priorité : Haute
- Difficulté : Haute

2.11. Gestion des animations

Une animation se déclenche à l'obtention d'une étoile ou quand un bâtiment est détruit pour mieux indiquer visuellement la situation au joueur.

- Priorité : Moyenne
- Difficulté : Moyenne

2.12. Interface utilisateur

L'interface utilisateur proposera soit de former ses troupes, et d'attaquer quand les troupes seront prêtes. La formation des troupes sera représentée par trois boutons distincts représentant les troupes. Il y aura également un bouton Attaquer pour lancer une attaque sur un village.

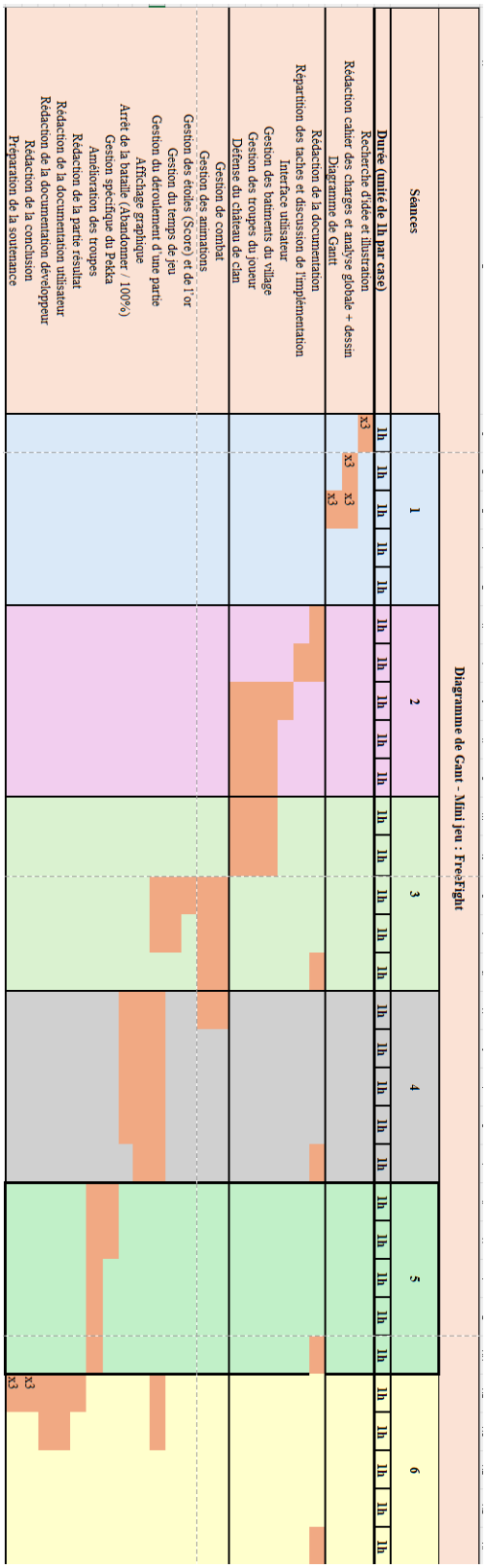
- Priorité : Moyenne
- Difficulté : Moyenne

2.13. Gestion du temps de jeu

Au début de chaque partie, un timer de 2 minutes se déclenche lors du déploiement de la première troupe. Le timer s'écoule jusqu'à la fin ou quand la partie est interrompue.

- Priorité : haute
- Difficulté : Moyenne

3. Plan de développement



4. Conception générale

4.1. Structure du jeu

Le projet est structuré selon une architecture MVC.

Le modèle regroupe les classes Troupe, Barbare, Sorcier, Pekka, Batiment, Defense, Chateau, Partie, Ameliorations, Sauvegarde et Camp, et est responsable de la logique métier du jeu.

Il gère notamment :

- Le comportement des troupes (déplacement, attaque, mort),
- La gestion des bâtiments et des défenses,
- Le déroulement d'une partie (score, temps, conditions de victoire),
- Les améliorations des unités ainsi que la sauvegarde de la progression.

La classe Partie joue un rôle central car elle contient l'état complet du jeu et orchestre les interactions entre les différents éléments.

La vue comprend les classes Affichage, StartPanel, AmeliorationPanel et FinPanel, qui assurent l'affichage graphique du jeu.

- Dessiner les troupes, bâtiments et effets visuels,
- Afficher les informations de jeu (score, chrono, étoiles),
- Proposer les différentes interfaces (menu, améliorations, écran de fin).

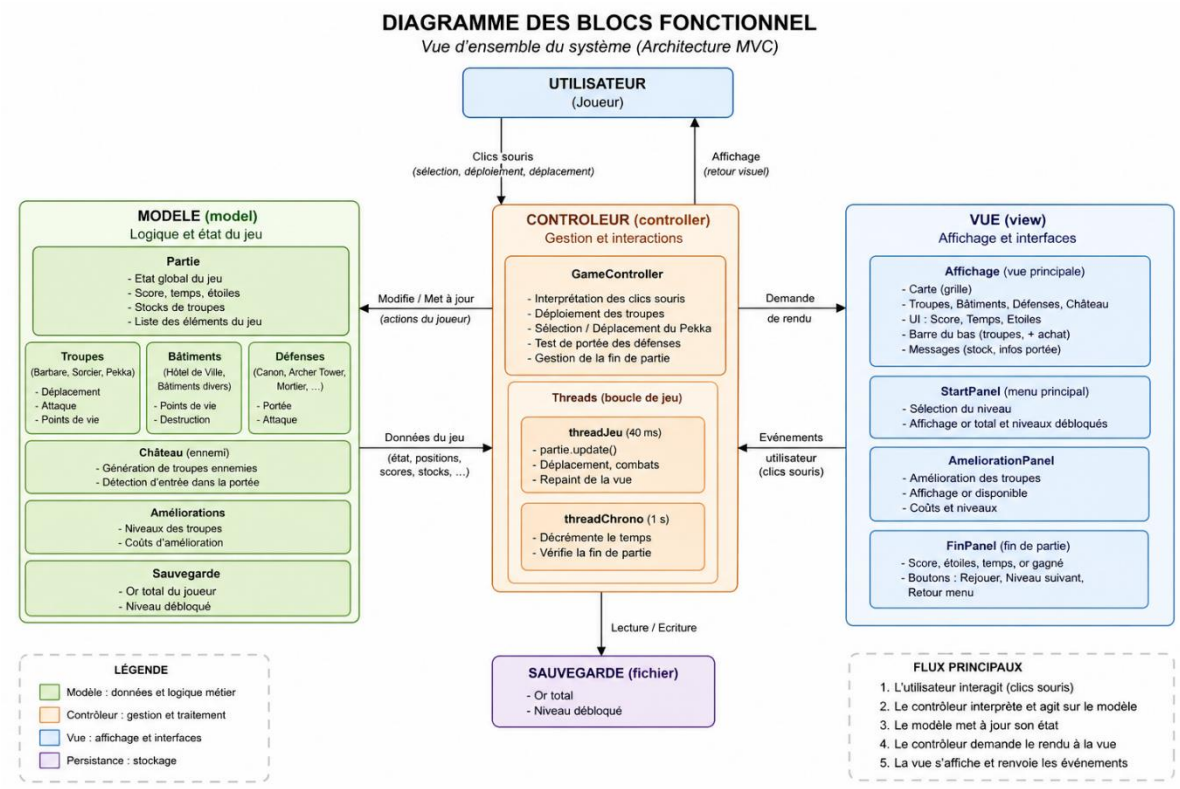
Elle ne contient aucune logique métier et se contente de représenter l'état du modèle.

Le **contrôleur** est représenté par la classe GameController.

Il fait le lien entre la vue et le modèle. Il interprète les clics de l'utilisateur, déclenche les actions sur le modèle et gère maintenant les deux threads principaux du jeu.

Enfin, la classe Test (main) permet d'initialiser l'application et de gérer la navigation entre les différentes interfaces (menu, améliorations, jeu, fin de partie).

4.2. Blocs fonctionnels et communication



4.3. Explication de l'architecture

Le lancement du jeu est assuré par la classe Test (main) qui initialise les différents composants nécessaires au fonctionnement de l'application.

L'initialisation crée :

- Une instance de Partie (modèle),
- Une instance de GameController (contrôleur),
- Une instance de Affichage (vue).

Le contrôleur est ensuite relié à la vue grâce à la méthode setAffichage(). Il ajoute un MouseListener sur la carte afin de détecter les clics de l'utilisateur.

Les clics sont interprétés selon plusieurs cas :

- Achat d'une troupe avec l'or disponible ;
- Sélection d'un type de troupe dans la barre du bas ;
- Sélection d'un Pekka déjà déployé ;
- Déplacement manuel du Pekka sélectionné ;
- Déploiement d'une troupe sur la carte ;
- Test de portée des défenses si aucun élément n'est sélectionné.

Le contrôleur gère aussi la fin de partie grâce à un callback appelé lorsque la partie est terminée. Cela permet à la classe Test d'afficher l'écran de fin.

La vue affiche en continu l'état du modèle et ne contient aucune logique métier.

4.4. Threads principaux

Le fonctionnement dynamique du jeu repose maintenant sur deux threads principaux, créés et gérés dans la classe GameLoop, instanciée par GameController.

- **threadJeu** :

Ce thread représente la boucle principale du jeu. Il s'exécute toutes les 40 ms environ. À chaque itération, il appelle partie.update() afin de faire évoluer le modèle : déplacement des troupes, attaques, génération d'ennemis, calcul du score et suppression des troupes mortes. Il déclenche ensuite le rafraîchissement graphique avec SwingUtilities.invokeLater().

- **threadChrono** :

Ce thread gère le temps de la partie. Il attend une seconde entre chaque exécution, puis décrémente le temps restant avec partie.decrementerTemps().
Il permet aussi de détecter la fin de partie lorsque le temps est écoulé.

Les deux threads utilisent la variable running pour savoir s'ils doivent continuer à fonctionner. Cette variable est déclarée en volatile, car elle est partagée entre plusieurs threads. La variable finNotifiee permet d'éviter d'appeler plusieurs fois la fin de partie.

Enfin, les appels à l'affichage sont faits avec SwingUtilities.invokeLater() afin de respecter le fonctionnement de Swing, puisque les modifications graphiques doivent être exécutées dans le thread graphique principal.

5. Conception détaillée

5.1. Création des troupes du joueur

5.1.1. Structure de données utilisée

Ceci est implémenté avec une classe principale Troupe.java contenant les attributs et méthodes communes, et trois autres classes qui héritent de Troupe.java qui sont :

- Sorcier.java
- Pekka.java
- Barbare.java

Les attributs des troupes sont :

- La position x, la position y, health (Points de vie), damage (dégâts que la troupe inflige), speed (la vitesse de déplacement de la troupe).

5.1.2. Conditions limites

La classe Troupe ne peut être instanciée directement (pas de troupes génériques), c'est pour cela qu'elle a été déclarée abstraite.

Chaque sous-classe définit ses propres caractéristiques.

5.1.3. Interactions avec les autres fonctionnalités

La classe affichage utilise les getters.

Les attributs health, damage, speed seront utilisés plus tard pour la gestion des combats, déplacement etc…

5.2. Affichage graphique

5.2.1. Structure de données utilisée

Ceci est implémenté avec une classe Affichage.java, elle hérite de Jpanel et fait partie du package view. Elle s'occupe de :

- Affichage des troupes : elle affiche la liste des troupes transmises par le model (Barbare, Pekka, Sorcier)

5.2.2. Constantes

```
int avatarSize = 50;  // Taille des avatars en bas de l'écran

int spacing = 120;                // Espacement horizontal entre deux avatars

int totalWidth = spacing * 2;            // Position verticale des avatars
```

5.2.3. Conditions limites

- Avatars visibles quel que soit la taille de la fenêtre
- Ils sont centrés horizontalement.
- Chaque type est affiché une seule fois.

5.2.4. Algorithme

1- Charger les images des troupes.
2- Définir la couleur du fond de terrain.
3- A chaque appel de paint
    - Effacer l'écran avec super.paint
    - Calculer la position horizontale de départ pour centrer les avatars.
    - Dessiner chaque avatar en bas de l'écran avec une taille réduite.
    - Afficher le nom correspondant à chaque image.

5.2.5. Interactions avec les autres fonctionnalités

- La classe reçoit la liste des troupes depuis la classe Test.
- Elle accède aux données des troupes avec leurs getters.
- Elle interagit avec le contrôle, la sélection de troupes et les animations graphiques.

Mise à jour :

Nous avons adapté la classe Affichage.java pour qu'elle soit compatible avec le comportement automatique des troupes.

Dans la première version, le déplacement était déclenché manuellement par un clic. Cette logique a été modifiée afin que l'actualisation du jeu se fasse automatiquement grâce à un Timer Swing. À chaque itération :

- Chaque troupe exécute sa méthode agir(...),
- Sa position est mise à jour,
- L'écran est redessiné avec repaint().

Enfin, l'affichage des bâtiments a été adapté pour ne plus afficher ceux qui sont détruits, ce qui permet de garder une cohérence entre les données du modèle et ce qui est affiché à l'écran.

La classe Affichage.java interagit directement avec :

- La liste des troupes du joueur, pour afficher leur position et leur évolution ;
- Les bâtiments et défenses, pour les dessiner et mettre à jour leur état ;
- La logique de jeu du modèle, via l'appel régulier de la méthode agir(...) ;
- Les futures animations et la gestion de combat, qui pourront s'appuyer sur cette boucle de mise à jour.

5.3. Gestion des troupes du joueur

On a d'abord complété la classe abstraite Troupe.java en ajoutant la logique de déplacement et de ciblage. Chaque troupe peut désormais :

- Rechercher une cible en fonction d'un ordre de priorité,
- Choisir le bâtiment le plus proche parmi les cibles possibles,
- Se déplacer progressivement vers cette cible selon sa vitesse,
- Attaquer le bâtiment une fois arrivée,
- Recalculer une nouvelle cible lorsque la précédente est détruite.

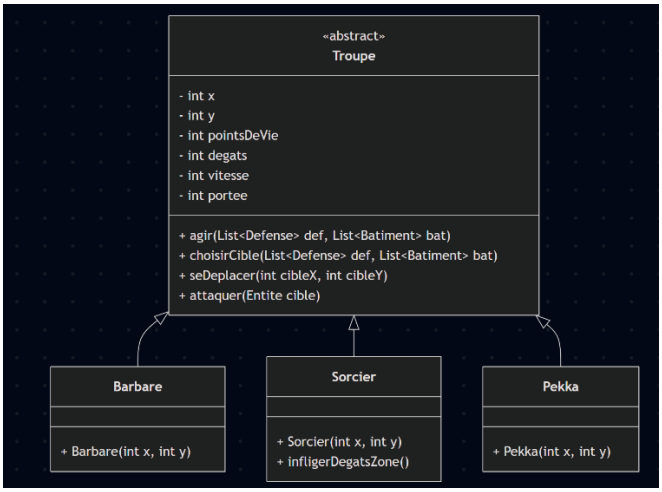
Les principales méthodes utilisées pour cela sont :

- moveTo(int targetX, int targetY)
- isArrived(int targetX, int targetY)
- trouverPlusProche(List<Batiment> liste)
- choisirCible(List<Batiment> defenses, Batiment chateauDeClan, List<Batiment> autresBatiments)
- agir(List<Batiment> defenses, Batiment chateauDeClan, List<Batiment> autresBatiments)

5.3.1. Interactions avec les autres fonctionnalités

Le travail réalisé sur les troupes interagit avec plusieurs parties du projet :

- Avec les bâtiments, car les troupes utilisent leurs positions pour calculer la cible la plus proche
- Avec la gestion des combats, car les attributs damage et health sont utilisés lors des attaques ;
- Avec l'affichage graphique, car les positions des troupes sont mises à jour en continu puis redessinées à l'écran ;
- Avec le déroulement global d'une partie, car les troupes enchaînent déplacement, attaque et changement de cible.



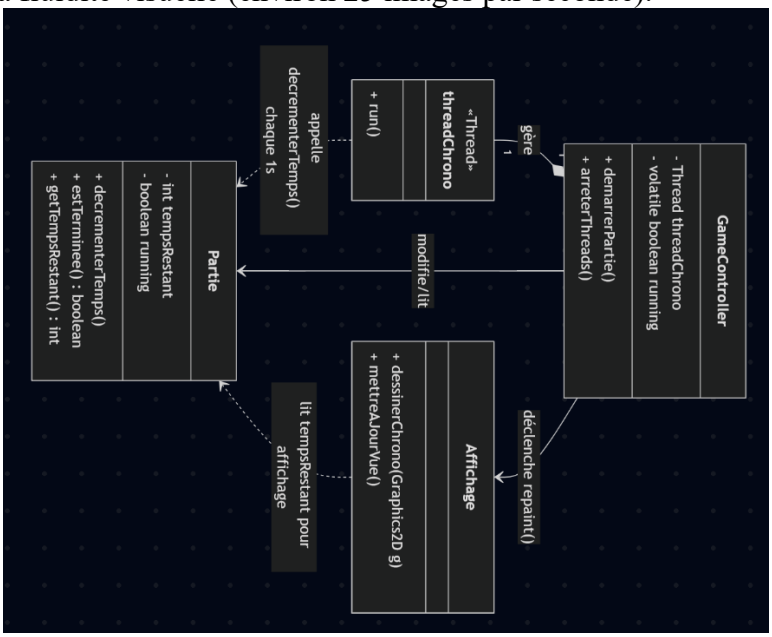
5.4. Gestion du temps de jeu

5.4.1. Structures de données et constantes

La gestion du temps de jeu est centralisée dans la classe Affichage, qui pilote le déroulement de la partie via deux constantes et deux timers distincts. La donnée maîtresse est l'attribut secondesRestantes, initialisé par une constante à 120 secondes, qui sert de référence pour la condition de victoire ou de défaite. Le mécanisme repose sur le découplage de deux flux temporels : un premier timer chrono, cadencé par une constante de 1000 ms, assure la logique métier en décrémentant le compteur chaque seconde et en déclenchant l'arrêt du jeu à zéro ; parallèlement, un second timer de 40 ms assure la fluidité visuelle en rafraîchissant l'interface pour animer les troupes et mettre à jour l'affichage dynamique du chronomètre.

Voici les constantes utilisées pour cette structure :

- TEMPS_INITIAL = 120 : Représente la durée totale de la partie en secondes.
- DELAI_CHRONO = 1000 : Définit l'intervalle d'une seconde pour le décompte du temps réel.
- DELAI_ANIMATION = 40 : Définit l'intervalle de rafraîchissement graphique pour assurer la fluidité visuelle (environ 25 images par seconde).



5.4.2. Algorithme

Le fonctionnement du chronomètre se déroule selon les étapes suivantes :

1. Initialisation : à la construction de Affichage, le Timer chrono est créé avec une action associée (lambda), mais il n'est pas encore démarré. secondesRestantes est positionné à 120.

2. Déclenchement : un clic sur le bouton « ⚔ Déployer » appelle demarrerChrono(). Cette méthode vérifie d'abord que le chrono n'est pas déjà actif via chrono.isRunning(). Si ce n'est pas le cas, elle appelle chrono.start().

3. Tick périodique : à chaque seconde, la lambda du Timer est exécutée :
   - si secondesRestantes > 0 : décrémentation de secondesRestantes, puis appel de repaint()
   - si secondesRestantes == 0 : appel de chrono.stop() pour stopper le timer, puis un dernier repaint() pour afficher l'état final.

4. Rendu du chrono : repaint() déclenche paintComponent() dans MapPanel, qui appelle dessinerChrono(Graphics2D g2). Cette méthode :
   - convertit secondesRestantes en format minutes:secondes via String.format("%d:%02d", minutes, secondes) ;
   - détermine l'état courant : normal (> 30 s), urgent (≤ 30 s), fini (= 0 s) ;
   - adapte la couleur du fond et du texte en conséquence (noir → orange → rouge)
   - affiche le message « Temps écoulé ! » lorsque secondesRestantes == 0.

5.4.3. Conditions limites

Les conditions suivantes doivent être respectées pour garantir le bon fonctionnement de la fonctionnalité, et constituent autant de cas à tester :

- <u>Unicité du démarrage</u> : demarrerChrono() ne doit démarrer le timer que si chrono.isRunning() retourne false. Un double-clic sur « Déployer » ne doit pas relancer ou accélérer le chrono. Ce cas est géré par la garde if (!chrono.isRunning()).

- <u>Plancher à zéro</u> : secondesRestantes ne doit jamais passer en dessous de 0. La condition if (secondesRestantes > 0) dans le listener assure que la décrémentation s'arrête à 0, et que chrono.stop() est appelé immédiatement après.

- <u>Arrêt du timer en fin de partie</u> : le timer doit s'arrêter lorsque secondesRestantes atteint 0. Dans l'état actuel du code, c'est la seule condition d'arrêt automatique ; les conditions liées à la destruction des bâtiments ou à la mort des troupes ne sont pas encore implémentées et devront appeler chrono.stop() explicitement.

- <u>Thread Swing</u> : repaint() et toutes les modifications de secondesRestantes s'exécutent dans le thread de l'Event Dispatch Thread (EDT) de Swing, puisque javax.swing.Timer garantit que son action s'exécute sur l'EDT. Cela assure qu'il n'y a pas de conflit d'accès concurrent entre le timer du chrono et celui du déplacement des troupes.

5.4.4. Interactions avec les autres fonctionnalités

La gestion du temps s'interface avec les autres composants du système de la façon suivante :

Module de déploiement : le bouton « ⚔ Déployer », défini dans le constructeur de Affichage, est le seul point d'entrée pour déclencher le chrono. Il appelle directement demarrerChrono(). Toute logique de déploiement des troupes qui viendrait s'y greffer devra passer par ce même point d'entrée.

Affichage graphique : dessinerChrono() est appelée à chaque cycle de rendu depuis paintComponent(), aux côtés de dessinerGrille(), dessinerDefenses(), dessinerTroupes(), etc. Elle utilise secondesRestantes, qui est un attribut partagé de Affichage : toute modification de cet attribut depuis une autre partie du code sera automatiquement reflétée au prochain repaint().

Timer de déplacement : le Timer timer (40 ms) tourne en parallèle du chrono et est indépendant de lui. Ces deux boucles ne se synchronisent pas explicitement, mais comme tous deux s'exécutent sur l'EDT, il n'y a pas de risque de concurrence.

Conditions de fin de partie : lorsque les modules de combat détecteront la destruction de tous les bâtiments ou l'élimination de toutes les troupes, ils devront appeler chrono.stop() directement sur l'instance, puis déclencher un repaint() pour mettre à jour l'écran. secondesRestantes pourra également être lu par ces modules pour conditionner certains comportements (bonus de temps, score, etc.).

5.5. Gestion spécifique du Pekka

5.5.1. Structure de données

La gestion du déplacement manuel du Pekka est implémentée dans la classe Pekka.java, cette classe hérite de la classe abstraite Troupe.java.

Contrairement aux autres troupes, le Pekka possède un comportement spécifique permettant à l'utilisateur de le contrôler manuellement. Pour cela, deux attributs supplémentaires ont été introduits :

- targetX : coordonnées horizontale de destination.
- targetY : coordonnées verticale de destination.

Ces attributs permettent de stocker une position cible définie par un clic utilisateur sur la carte.

Une méthode spécifique setDestination(int x, int y) permet d'affecter ces coordonnées lors de l'interaction avec l'utilisateur. Cette méthode annule également toute cible automatique (bâtiment) afin de privilégier le déplacement manuel.

5.5.2. Les constantes

Le Pekka ne nécessite pas de constantes spécifiques supplémentaires pour son déplacement.
Cependant, il utilise les constantes héritées de la classe Troupe :

- speed : vitesse de déplacemet
- damage : dégâts infligés
- health : points de vie

5.5.3. Conditions limites

Plusieurs cas limites ont été pris en compte :

- Si aucune destination n'est définie (targetX et targetY null), le Pekka adopte un comportement classique (attaque automatique).
- Si la destination est atteinte, les coordonnées sont réinitialisées afin d'éviter un déplacement infini.
- Si une cible (bâtiment) est définie, le Pekka peut attaquer normalement une fois arrivée.
- Le déplacement manuel ne s'applique qu'au Pekka et non aux autres troupes.

5.5.4. Algorithme

1 - L'utilisateur sélectionne le Pekka via un clic.

2 - L'utilisateur clique sur une position de la carte.

3 - On enregistre la destination.
4 - À chaque mise à jour du jeu :

- Si une destination est définie :
  - Le Pekka se déplace progressivement vers celle-ci…
  - Une fois arrivé, la destination est supprimée.

- Sinon :
  - Le comportement standard est appliqué :
    déplacement vers une cible (bâtiment) et attaquer une fois arrivé.

5.5.5. Interactions avec les autres fonctionnalités

La gestion du Pekka interagit avec plusieurs composants du projet :

- Troupe.java : fournit les attributs et les méthodes principales.
- Controlleur : pour la détection de clics à la sélection et pour le déplacement.

5.6. Gestion du combat

5.6.1. Structure de données utilisée

Le système de combat est géré principalement par la classe **Partie.java**, qui centralise l'ensemble des interactions entre les troupes et les bâtiments ennemis.

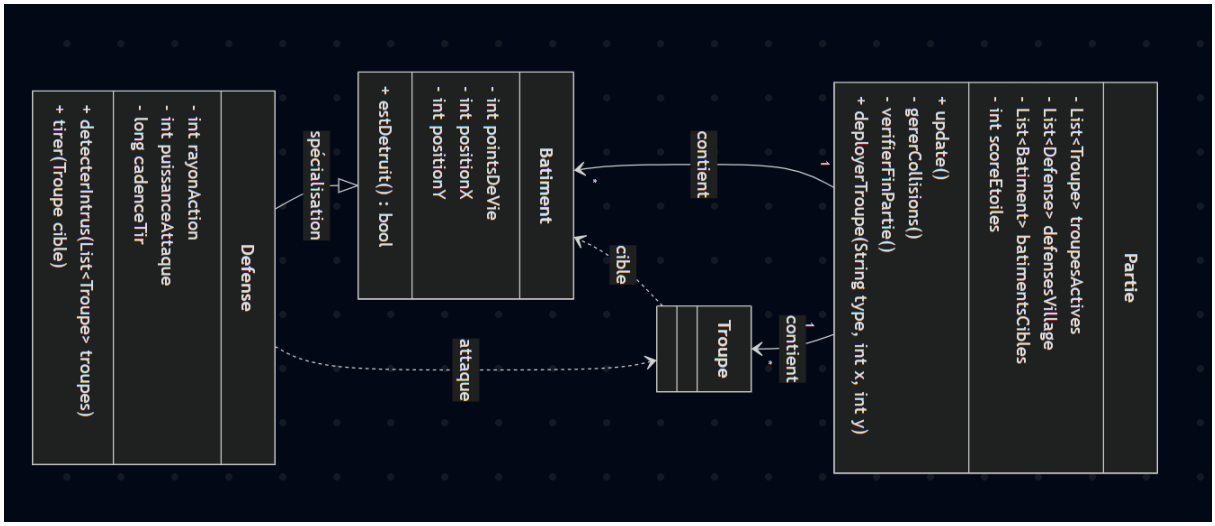
Le combat repose sur plusieurs structures de données :

- Une liste de troupes (List<Troupe>) représentant toutes les unités présentes sur la carte,
- Une liste de défenses (List<Defense>) représentant les bâtiments offensifs ennemis,
- Une liste de bâtiments (List<Batiment>) représentant les structures destructibles,
- Une instance de Chateau permettant de générer des troupes ennemies dynamiquement.

Chaque troupe possède :

- Une cible bâtiment,
- Une cible troupe ennemie prioritaire.

Le système de combat repose donc sur une organisation orientée objet où chaque entité (troupe ou bâtiment) possède ses propres comportements.



5.6.2. Les constantes

Le système de combat utilise plusieurs constantes définies dans différentes classes :

- Dans la classe Troupe.java :

health : points de vie de la troupe
damage : dégâts infligés
speed : vitesse de déplacement

- Dans la classe Defense.java :

portee : rayon d'attaque
degats : dégâts infligés par tir
cadenceTir : fréquence des tirs

Ces constantes permettent de définir le comportement et l'équilibrage du combat.

5.6.3. Conditions limites

Plusieurs cas limites ont été pris en compte :

- Si une troupe n'a aucune cible valide, elle ne se déplace pas.
- Si une cible est détruite, une nouvelle cible est automatiquement recherchée.
- Si une troupe ennemie est présente, elle est prioritaire sur les bâtiments.
- Une défense ne peut attaquer que si :
  - La cible est dans sa portée,
  - Le délai de tir (cadence) est respecté.
- Les troupes mortes ne peuvent plus être ciblées ni agir.
- Une troupe est supprimée seulement après son animation de mort.

- Le château ne génère ses troupes qu'une seule fois lorsque des troupes ennemies entrent dans sa portée.

5.6.4. Algorithme

Le système de combat est exécuté à chaque mise à jour du jeu (update() dans Partie.java) :

1 - Vérifier si le château doit générer des troupes ennemies
2 - Assigner une cible ennemie à chaque troupe
3 - Pour chaque troupe :
  - Si une troupe ennemie est ciblée :
  - se déplacer vers elle
  - attaquer une fois à portée

  Sinon :
  - se déplacer vers un bâtiment
  - attaquer une fois arrivé

4 - Pour chaque défense :
  - Vérifier si une troupe est à portée
  - Attaquer selon la cadence de tir

5 - Mettre à jour le score si des bâtiments sont détruits
6 - Supprimer les troupes mortes après animation

5.6.5. Interactions avec les autres fonctionnalités

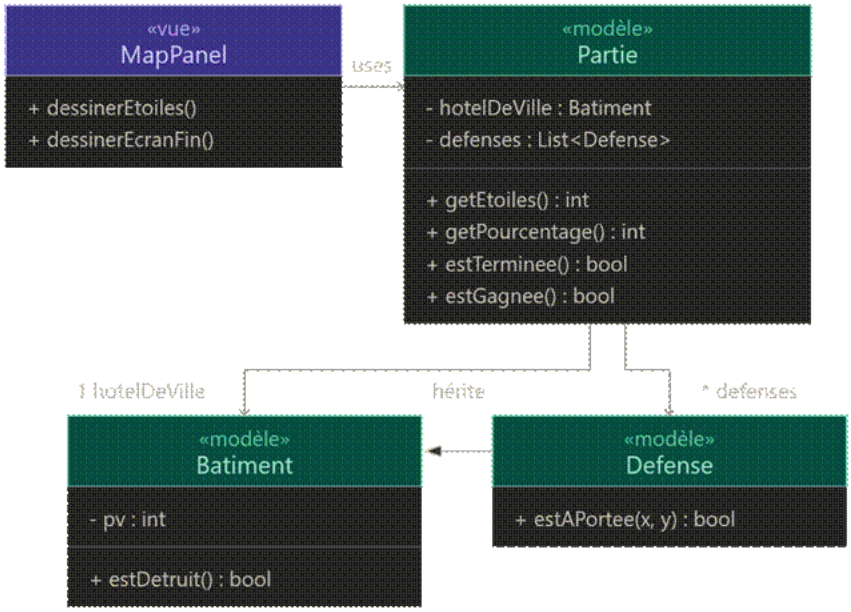
La gestion du combat interagit avec plusieurs composants du projet :
- Troupe.java : définit les comportements de base des unités (déplacement, attaque)
- Defense.java : gère les attaques à distance des bâtiments ennemis
- Chateau.java : génère des troupes ennemies pendant la partie
- Partie.java : coordonne toutes les interactions de combat
- GameController : déclenche les mises à jour du combat via la boucle de jeu

5.7. Gestion des étoiles et de la fin de partie

5.7.1 Structures de données et constantes

Cette fonctionnalité est portée conjointement par Partie.java (logique) et Affichage.java (rendu). Aucune variable ne stocke le nombre d'étoiles : il est calculé à la volée à partir de l'état des bâtiments. Les données impliquées sont :

- private Batiment hotelDeVille : sa destruction rapporte une étoile indépendamment du pourcentage global.
- private List<Defense> defenses et private List<Batiment> autresBatiments : leur état de destruction alimente le pourcentage. autresBatiments est vide pour l'instant mais déjà intégré dans les calculs.
- estDetruit() dans Batiment : indicateur binaire utilisé pour évaluer l'état de chaque structure.



5.7.2. Algorithme

Calcul du pourcentage (getPourcentageDestruction()) : on divise le nombre de bâtiments détruits par le total (1 + defenses.size() + autresBatiments.size()), multiplié par 100 (division entière).

Calcul des étoiles (getEtoiles()) : trois conditions indépendantes, chacune valant +1 étoile :

- l'hôtel de ville est détruit ;
- le pourcentage de destruction ≥ 50 % ;
- le pourcentage de destruction ≥ 100 %.

Affichage en jeu (dessinerEtoiles()) : appelée à chaque frame depuis paintComponent(). Dessine un fond arrondi centré en haut de l'écran, puis 3 étoiles en polygone à 10 sommets (5 extérieurs, 5 intérieurs). Les i < etoiles premières sont dorées, les autres grises. Le pourcentage est affiché en texte à droite.

Écran de fin (dessinerEcranFin()) : appelée en dernier dans paintComponent() uniquement si partie.estTerminee() est vrai, de sorte qu'elle s'affiche par-dessus tout. Elle plaque un fond semi-transparent sur tout l'écran, puis affiche un panneau central avec le titre « VICTOIRE » ou « DÉFAITE » (selon estGagnee()), les 3 étoiles en taille agrandie (40 px au lieu de 28), et le pourcentage de destruction.

5.7.3. Conditions limites

- Division par zéro : si total == 0 dans getPourcentageDestruction(), une erreur se produit. En pratique hotelDeVille est toujours présent (total >= 1), mais ce cas devra être protégé si la configuration évolue.

- Indépendance des conditions d'étoiles : les trois critères sont évalués séparément. Il faut vérifier que la destruction de l'hôtel de ville seul ne déclenche pas la condition des 50 % si ce bâtiment représente moins de la moitié du total.
- Double condition de fin : estTerminee() est vrai si le temps est écoulé ou si 100 % des bâtiments sont détruits. Les deux cas doivent être testés pour confirmer que l'écran de fin s'affiche correctement dans chacun.
- Cas 0 étoile : si le temps s'écoule sans aucune destruction, l'écran de fin doit s'afficher en mode « DÉFAITE » avec 0% détruit et 3 étoiles grises.
- Irréversibilité : une fois estTerminee() vrai, l'écran de fin s'affiche à chaque frame. Il n'existe pas encore de mécanisme de retour au menu ; cela devra être ajouté et testé.

5.7.4. Interactions avec les autres fonctionnalités

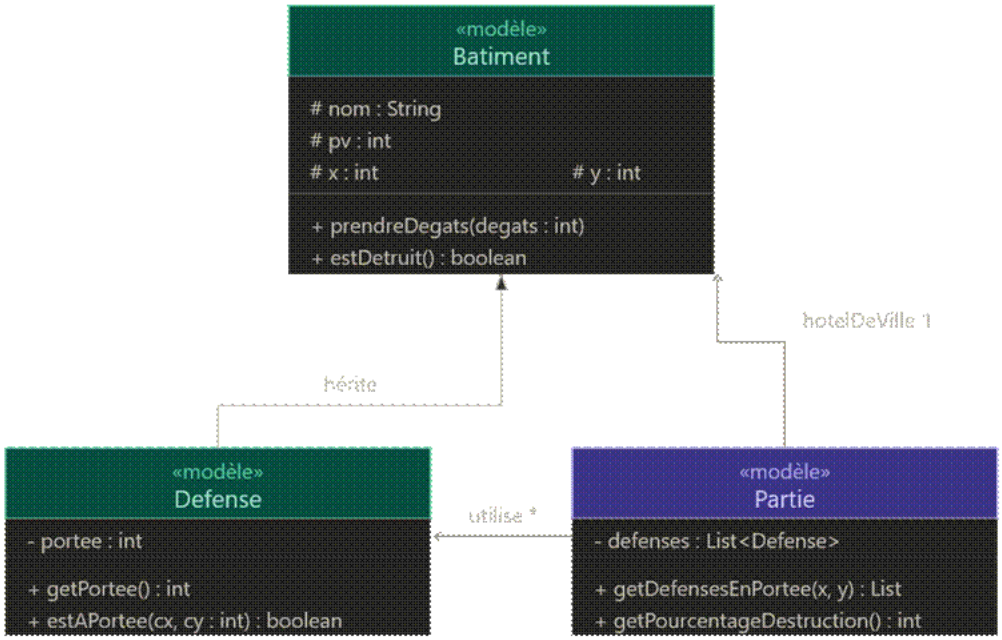
- Module de combat : seul module qui réduit les PV des bâtiments. Chaque bâtiment tombé à 0 PV fait basculer estDetruit() à true, impactant immédiatement le pourcentage et les étoiles au prochain rendu.
- Chronomètre : tempsEcoule() est l'une des deux conditions de estTerminee(), et donc de déclenchement de l'écran de fin. Les deux fonctionnalités convergent vers ce même point.
- Boucle de rendu : le timer de déplacement (40 ms) continue d'appeler partie.update() même après la fin de la partie. Il devra être conditionné par !partie.estTerminee() pour éviter des traitements inutiles une fois la partie terminée.
- Affichage : dessinerEtoiles() et dessinerEcranFin() sont appelées à chaque cycle de paintComponent(), garantissant une mise à jour en temps réel sans aucun stockage intermédiaire du score.

5.8. Gestion des bâtiments et des défenses

5.8.1. Structures de données et constantes

Cette fonctionnalité repose sur deux classes du package model :

- Batiment.java : classe parente représentant tout bâtiment du village. Ses quatre attributs protected sont String nom, int pv, int x et int y. Elle expose des getters pour chacun, une méthode prendreDegats(int degats) qui réduit les PV avec un plancher à 0, et estDetruit() qui retourne pv <= 0.
- Defense.java : hérite de Batiment et ajoute un attribut private int portee, accessible via getPortee(). Elle ajoute également la méthode estAPortee(int cx, int cy).



5.8.2. Algorithme

Calcul de portée (estAPortee(int cx, int cy)) : on calcule la distance euclidienne entre le centre de la défense (x, y) et le point testé (cx, cy), puis on vérifie si elle est inférieure ou égale à portee :

distance = sqrt($(cx - x)^2 + (cy - y)^2$)
retourner distance <= portee

Prise de dégâts (prendreDegats(int degats)) : on soustrait les dégâts aux PV courants. Si le résultat passe en dessous de 0, les PV sont ramenés à 0 exactement, ce qui garantit qu'estDetruit() retourne true sans valeur négative.

Détection des défenses en portée : Partie.getDefensesEnPortee(x, y) parcourt la liste defenses et appelle estAPortee() sur chacune, retournant la sous-liste des défenses qui couvrent le point testé.

Affichage : chaque défense dessine en permanence son cercle de portée en pointillés jaunes. Si elle figure dans defensesEnPortee, son cercle passe en rouge et un bandeau s'affiche en haut de l'écran. L'hôtel de ville se distingue visuellement par son toit triangulaire. Les bâtiments détruits (estDetruit() == true) ne sont plus dessinés.

5.8.3. Conditions limites

- prendreDegats() garantit que pv ne passe jamais en dessous de 0 : le plancher est explicitement appliqué (if (pv < 0) pv = 0), ce qui rend estDetruit() fiable.
- estDetruit() est le seul point de sortie pour déterminer qu'un bâtiment est hors jeu ; tout module qui souhaite vérifier l'état d'un bâtiment doit passer par cette méthode et non comparer pv directement.
- Si portee == 0, estAPortee() ne retournera true que si la troupe est exactement sur le centre de la défense - cas à éviter dans la configuration des données.

- L'ordre des paramètres du constructeur de Defense est (nom, pv, portee, x, y) - différent de Batiment qui est (nom, pv, x, y) - ce qui peut induire des erreurs à l'instanciation si l'on confond les deux.

5.8.4. Interactions avec les autres fonctionnalités

Système d'étoiles : estDetruit() est appelée par getPourcentageDestruction() dans Partie pour compter les bâtiments détruits et calculer le score.

Module de combat : les troupes appellent prendreDegats() pour infliger des dégâts, et estAPortee() pour vérifier si une défense peut les cibler.

Boucle de jeu : getDefensesEnPortee() est appelée dans Partie à chaque mise à jour pour évaluer quelles défenses sont activées.

Affichage : dessinerDefenses() et dessinerHotelDeVille() dans MapPanel lisent les getters de Batiment et Defense (getNom(), getPv(), getX(), getY(), getPortee()) à chaque frame pour redessiner l'état courant.

5.9. Défense du château de clan

5.9.1. Structure de données utilisée

La gestion de la défense du château de clan est implémentée dans la classe Chateau.java, qui hérite de la classe Defense.java.

Cette classe permet de représenter un bâtiment spécial capable de générer des troupes ennemies.

Elle contient un attribut spécifique :
aSpawn : booléen indiquant si les troupes ont déjà été générées.

Le château utilise également une liste de troupes (List<Troupe>) pour ajouter les unités générées directement dans le jeu.

Les troupes créées sont de différents types (Barbare, Sorcier, Pekka) et sont automatiquement assignées au camp ennemi.

5.9.2. Les constantes

Le château de clan ne possède pas de constantes spécifiques supplémentaires.
Cependant, il utilise les constantes héritées de la classe Défense :

- portee : rayon de détection des troupes ennemies
- cadenceTir : fréquence d’action
- degats : dégâts potentiels

Il utilise également les caractéristiques des troupes générées :

- Points de vie
- Dégâts
- Vitesse

5.9.3. Conditions limites

Plusieurs cas limites ont été pris en compte :

- Le spawn des troupes ne peut se produire qu'une seule fois grâce à la variable aSpawn.
- Si aucune troupe du joueur n'entre dans la zone de détection, aucune troupe ennemie n'est générée.
- Les troupes générées sont automatiquement placées autour du château pour éviter la superposition.
- Les troupes ennemies appartiennent au camp ENNEMI et n'attaquent que les troupes du joueur.
- Si le château est détruit avant activation, le spawn ne peut pas se produire.

5.9.4. Algorithme

1 - Vérifier si le château a déjà généré ses troupes
2 - Parcourir les troupes du joueur
3 - Si une troupe entre dans la portée du château :
  - Générer plusieurs troupes ennemies autour du château
  - Affecter leur camp à ENNEMI
  - Les déployer sur la carte
  - Marquer le spawn comme effectué
4 - Les troupes générées :
  - Ciblent les troupes du joueur
  - Se déplacent vers elles
  - Les attaquent automatiquement

5.9.5. Interactions avec les autres fonctionnalités

La défense du château de clan interagit avec plusieurs composants du projet :

- Partie.java : déclenche la génération des troupes lors de la mise à jour du jeu
- Troupe.java : gère le comportement des troupes générées (déplacement, attaque)
- Camp.java : permet de distinguer les troupes alliées et ennemies
- Defense.java : fournit les mécanismes de portée et de détection
-

5.10. Gestion des niveaux

5.10.1 Structure de données utilisée

La gestion des niveaux est répartie entre plusieurs classes du projet.

La classe StartPanel.java, située dans le package view, correspond à l'écran de sélection des niveaux. Elle affiche les différentes cartes de niveaux disponibles. Chaque carte contient le nom du niveau, quelques informations sur sa difficulté, ainsi qu'un bouton pour lancer la partie.

Cette classe utilise les informations venant de Sauvegarde.java afin de savoir quels niveaux sont déjà débloqués. Ainsi, si un niveau n'est pas encore accessible, son bouton est désactivé.

La classe Partie.java, située dans le package model, contient la configuration réelle de chaque niveau. Elle possède plusieurs méthodes comme :

- configurerNiveau1()
- configurerNiveau2()
- configurerNiveau3()
- configurerNiveau4()

Chaque méthode initialise une partie avec ses propres bâtiments, défenses, points de vie, temps de jeu et troupes disponibles.

La classe Sauvegarde.java permet de conserver la progression du joueur. Elle utilise un fichier appelé sauvegarde.dat. Ce fichier permet de stocker l'or total du joueur ainsi que le dernier niveau débloqué.

Le niveau choisi par le joueur est ensuite transmis depuis la classe Test.java vers la classe Partie.java. Selon le niveau sélectionné, la méthode de configuration correspondante est appelée.

5.10.2. Les constantes

- La progression du jeu repose sur 4 niveaux.

Constante	Valeur	Description
Niveau 1	3 défenses, 90 s	Goblin Village
Niveau 2	4 défenses, 105 s	Camp Barbare
Niveau 3	6 défenses, 120 s	Forteresse Sombre
Niveau 4	8 défenses, 105 s	Donjon du Champion

- Chaque niveau possède aussi ses propres valeurs pour les bâtiments principaux.

Niveau	PV Hôtel de Ville	PV Château de Clan	Troupes Pekka disponibles
Niveau 1	1 500	500	3
Niveau 2	2 000	700	3
Niveau 3	2 500	1 000	3
Niveau 4	3 000	1 500	2

Ces valeurs permettent d'augmenter progressivement la difficulté. Plus le joueur avance dans les niveaux, plus les bâtiments sont résistants et plus les défenses sont nombreuses.

5.10.3 Conditions limites

Plusieurs cas limites ont été pris en compte.

- Au début du jeu, seul le niveau 1 est accessible.
- Les autres niveaux se débloquent progressivement.
- Pour débloquer le niveau suivant, le joueur doit obtenir au moins une étoile dans le niveau précédent.
- Si un niveau est verrouillé, son bouton est désactivé.
- Un message indique au joueur qu'il doit terminer le niveau précédent avant de pouvoir y accéder.
- Le niveau 4 est plus difficile que les autres car il contient plus de défenses et réduit le nombre de Pekka disponibles.
- Le niveau 4 ajoute également un nouveau type de défense, appelé Inferno.
- À chaque nouvelle partie, les données du niveau sont recréées entièrement. Cela évite de garder l'état d'une ancienne partie.

5.10.4 Algorithme

Le fonctionnement de la gestion des niveaux se déroule de la manière suivante :

1 - Charger la progression depuis Sauvegarde.java.

2 - Récupérer :
    L'or total du joueur ;
    Le dernier niveau débloqué.

3 - Afficher les 4 cartes de niveau dans StartPanel.

4 - Pour chaque carte :
    Si le niveau est débloqué, le bouton Jouer est actif ;
    Si le niveau est verrouillé, le bouton est désactivé.

5 - Lorsque le joueur clique sur Jouer :
    Appeler Test.choisirNiveau(numNiveau) ;
    Afficher l'écran d'amélioration des troupes.

6 - Lorsque le joueur clique sur Lancer la bataille :
    Créer une nouvelle instance de Partie ;
    Appeler la méthode configurerNiveauN() correspondant au niveau choisi ;
    Créer le GameController ;
    Créer l'interface Affichage ;
    Lancer les threads du jeu.

7 - À la fin de la partie :
    Ajouter l'or gagné à la sauvegarde ;
    Vérifier le nombre d'étoiles obtenues ;
    Si le joueur a obtenu au moins une étoile, débloquer le niveau suivant.

5.10.5 Interactions avec les autres fonctionnalités

La gestion des niveaux interagit avec plusieurs parties du projet.

- Sauvegarde.java : fournit le niveau débloqué et l'or total au lancement du jeu. Elle est aussi mise à jour à la fin d'une partie.
- StartPanel.java : affiche les niveaux disponibles et bloque ceux qui ne sont pas encore accessibles.
- Partie.java : initialise les bâtiments, les défenses, le temps de jeu et les troupes selon le niveau choisi.
- AmeliorationPanel.java : apparaît après le choix du niveau et avant le lancement de la bataille.
- FinPanel.java : affiche le score, les étoiles et l'or gagné. Il permet aussi de rejouer, de passer au niveau suivant ou de retourner au menu.
- Test.java : fait le lien entre les différents écrans et transmet le niveau sélectionné.

5.11 Amélioration des troupes

5.11.1 Structure de données utilisée

L'amélioration des troupes est gérée par deux classes principales.

La classe Ameliorations.java, située dans le package model, contient les niveaux actuels des troupes ainsi que l'or disponible pour les améliorer. Elle permet de savoir si une amélioration est possible et de calculer son coût.

Les troupes concernées sont :
- Barbare
- Sorcier
- Pekka

Chaque troupe commence au niveau 1. Le joueur peut ensuite améliorer chaque type de troupe jusqu'au niveau maximum.

La classe AmeliorationPanel.java, située dans le package view, représente l'écran graphique affiché avant le début de la bataille. Cet écran permet au joueur de voir les caractéristiques de ses troupes et de dépenser son or pour les améliorer.

Les niveaux d'amélioration sont ensuite utilisés dans les classes Barbare.java, Sorcier.java et Pekka.java. Lorsqu'une troupe est créée, son constructeur reçoit son niveau et adapte automatiquement ses caractéristiques : points de vie, dégâts et vitesse.

5.11.2 Les constantes

Les constantes principales sont les suivantes :

Constante	Valeur	Description
OR_Depart	1 500	Or initial disponible pour les améliorations
NIVEAU_MAX	5	Niveau maximum pour chaque troupe

- Les coûts d'amélioration dépendent du type de troupe.

Troupe	Formule de coût	N2	N3	N4	N5
Barbare	100 × (niveau suivant - 1)	100 or	200 or	300 or	400 or
Sorcier	50 + 100 × (niveau suivant - 1)	150 or	250 or	350 or	450 or
Pekka	200 + 100 × (niveau suivant - 1)	300 or	400 or	500 or	600 or

5.11.3 Conditions limites

Plusieurs conditions limites sont prises en compte.

- Si le joueur n'a pas assez d'or, le bouton d'amélioration est désactivé.
- Lorsque le coût est trop élevé, il est affiché clairement pour montrer au joueur qu'il ne peut pas améliorer la troupe.
- Si une troupe atteint le niveau 5, elle ne peut plus être améliorée.
- Dans ce cas, le bouton affiche Niveau max.
- L'or de départ ne permet pas d'améliorer toutes les troupes au maximum.
- Le joueur doit donc faire des choix stratégiques entre les différentes unités.
- L'or gagné pendant les combats est sauvegardé grâce à Sauvegarde.java.
- Les niveaux des troupes sont réinitialisés au niveau 1 à chaque nouvelle partie. Cela permet de refaire des choix d'amélioration avant chaque bataille.

5.11.4 Algorithme

Le fonctionnement de l'amélioration des troupes se déroule ainsi :

1 - Charger l'or total depuis Sauvegarde.java.

2 - Créer une instance de Ameliorations.

3 - Initialiser :

- le niveau du Barbare à 1 ;
- le niveau du Sorcier à 1 ;
- le niveau du Pekka à 1 ;
- l'or disponible avec l'or sauvegardé.

4 - Afficher AmeliorationPanel.

5 - Pour chaque troupe :

- afficher son nom ;
- afficher son niveau actuel ;
- afficher ses points de vie ;
- afficher ses dégâts ;
- afficher sa vitesse ;
- calculer le coût du niveau suivant.

6 - Vérifier si l'amélioration est possible :

Si le niveau est égal à NIVEAU_MAX, désactiver le bouton ;
Si l'or est insuffisant, désactiver le bouton ;
Sinon, activer le bouton Améliorer.

7 - Lorsque le joueur clique sur Améliorer :

Appeler la méthode d'amélioration dans Ameliorations ;
Retirer le coût de l'or disponible ;
Augmenter le niveau de la troupe ;
Mettre à jour l'affichage.

8 - Lorsque le joueur clique sur Lancer la bataille :

Transmettre l'objet Ameliorations à Partie ;
Créer les troupes avec les niveaux choisis ;
Appliquer les nouvelles statistiques dans les constructeurs des troupes.

5.11.5 Interactions avec les autres fonctionnalités

L'amélioration des troupes interagit avec plusieurs composants du projet.

- Sauvegarde.java : fournit l'or disponible avant la bataille et enregistre l'or gagné à la fin.
- Ameliorations.java : calcule les coûts, vérifie si une amélioration est possible et stocke les niveaux des troupes.
- AmeliorationPanel.java : affiche les informations au joueur et permet de choisir les améliorations.
- Partie.java : reçoit les niveaux des troupes et les utilise lors de leur création.
- Barbare.java, Sorcier.java et Pekka.java : appliquent les formules de progression dans leur constructeur.
- FinPanel.java : affiche l'or gagné à la fin de la partie, qui pourra ensuite être utilisé pour les prochaines améliorations.

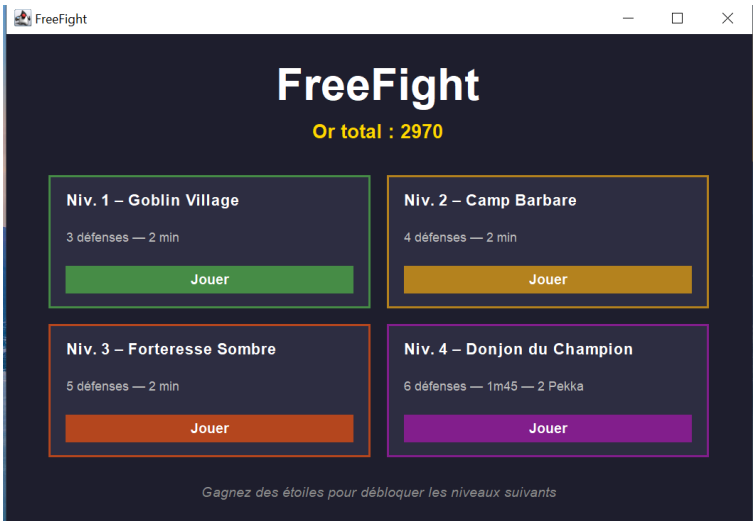
6. Résultat

La version finale de ce qu'on a pu coder jusqu'à présent propose une interface graphique dynamique et interactive, intégrant plusieurs modules qui coopèrent et offrent une expérience de jeu fluide sans interruption ni décalage.

L'interface se compose d'un écran de démarrage permettant de sélectionner un niveau, d'un écran d'amélioration des troupes, ainsi que d'une scène de jeu dans laquelle le joueur déploie ses unités sur un village ennemi.

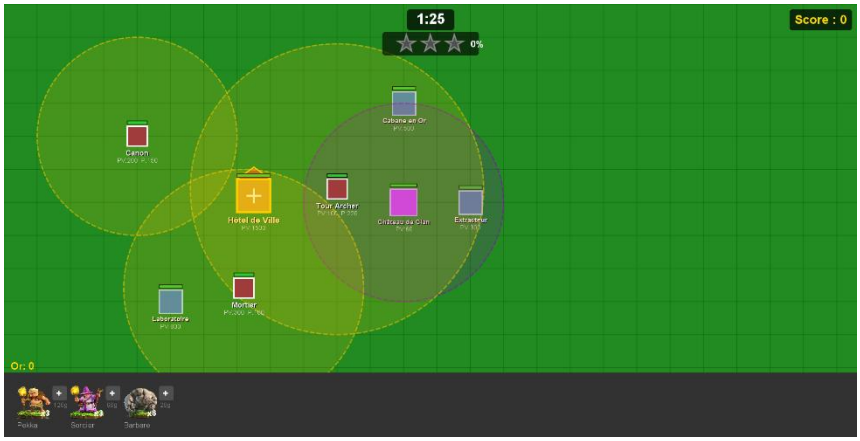
Le joueur interagit en temps réel avec l'environnement en sélectionnant des troupes, en les plaçant stratégiquement sur la carte et en observant leur comportement autonome (déplacement, attaque). Les combats se déroulent automatiquement entre les troupes et les défenses, tandis que le score, le pourcentage de destruction et les étoiles évoluent au fil de la partie.

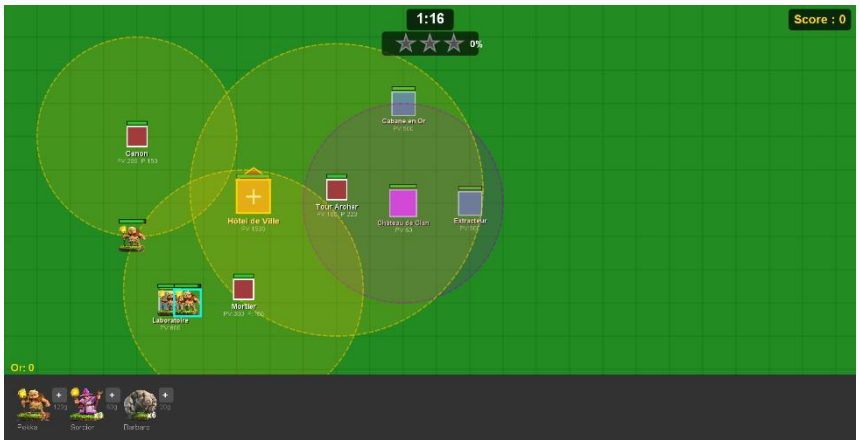
Ecran d'accueil



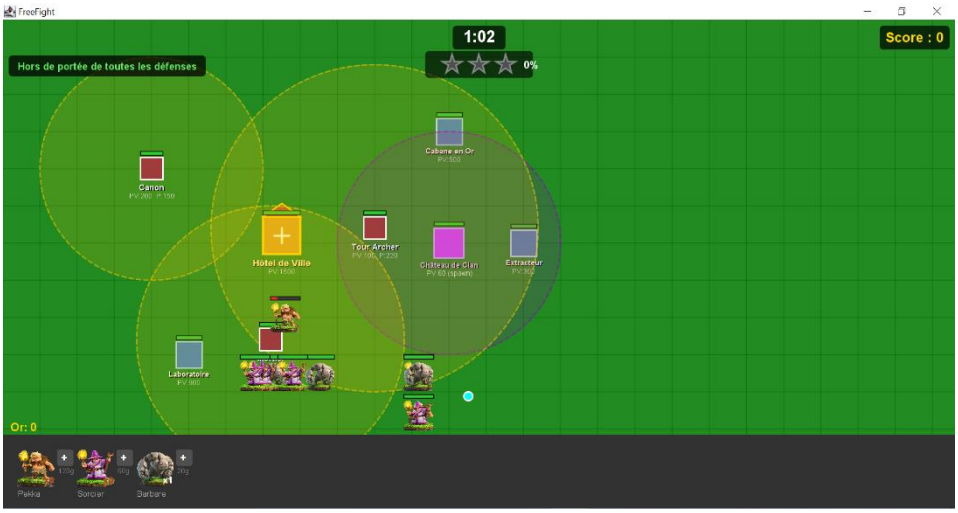
Après avoir choisi le niveau, on arrive sur le menu pour améliorer les troupes

Ensuite, on arrive sur la partie ou on peut déployer les troupes sur l'écran et le timer commence directement, et sur la deuxième photo on montre que les Pekka sont particuliers car on peut les sélectionner et leur donner une direction.

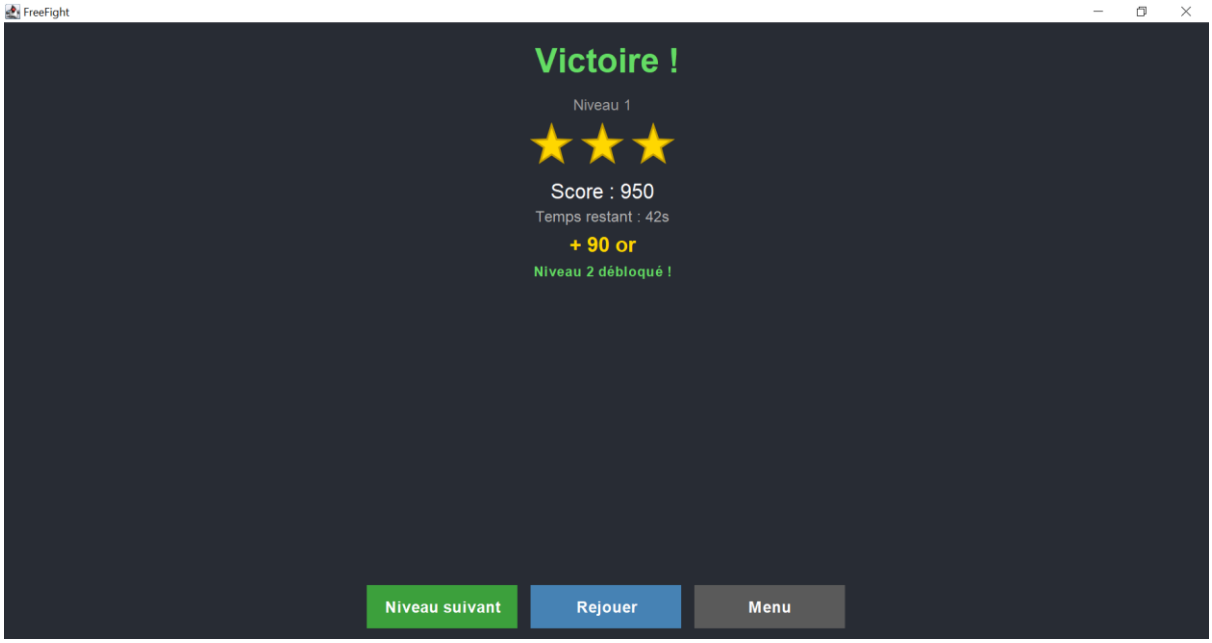




Des troupes ennemies sortent du château de clan pour combattre les troupes du joueur et leur baisser leurs points de vie.



Fin de partie après détruit le village ennemi et passé le niveau 1.



7. Documentation utilisateur

7.1 Pré Requis logiciels

Pour exécuter le jeu, les éléments suivants sont nécessaires :

- Java Development Kit (JDK)
- Un éditeur de texte et un terminal pour compiler et exécuter les fichiers sources (Eclipse par exemple).

Pas besoin de bibliothèques particulières car on utilise seulement SWING, AWT

7.2 Lancement du jeu

- Télécharger les dossiers regroupés sur ce repository git.
- Ouvrir les fichiers dans l'éditeur de texte.
- Exécuter le *main* se trouvant dans la classe test.java

7.3 Démarrer une partie

Au lancement du jeu, un menu principal s'affiche permettant de choisir un niveau.

Après sélection d'un niveau :

- Un écran d'amélioration des troupes apparaît,
- Le joueur peut améliorer ses unités avec l'or disponible,
- Puis cliquer sur "Lancer la bataille" pour commencer la partie.

L'écran de jeu s'affiche ensuite et la partie démarre automatiquement.

7.4. Commandes utilisateur

- Clic gauche sur un avatar (en bas) : sélectionner un type de troupe
- Clic gauche sur la carte : déployer une troupe à la position choisie
- Clic gauche sur le Pekka pour le déployer, ensuite clique gauche sur le Pekka déployé pour lui donner une direction si on veut.

7.5. Objectif du jeu

Le but du jeu est de détruire le village ennemi en utilisant les troupes disponibles pour accéder aux prochains niveaux

Le joueur doit :

- Détruire les bâtiments,
- Maximiser le score,
- Atteindre un pourcentage de destruction élevé,
- Obtenir jusqu'à 3 étoiles selon ses performances.

7.6. Fin de partie

La partie se termine lorsque :

- Le temps est écoulé,

- Ou lorsque tous les bâtiments sont détruits.

Un écran de fin s'affiche alors, indiquant :
- Le score,
- Le nombre d'étoiles obtenues,
- Le temps restant,
- L'or gagné.

Le joueur peut ensuite :
- Rejouer le niveau,
- Passer au niveau suivant (si débloqué)
- Ou retourner au menu principal.

8. Documentation développeur

Pour pouvoir prendre en main le code afin de l'améliorer ou ajouter des fonctionnalités, en addition à la partie conception, ce guide permet d'avoir une vue globale du projet.

8.1 Point d'entrée du programme

Le point d'entrée du projet se trouve dans la classe Test.java (package Main).

Cette classe initialise la fenêtre principale du jeu et gère la navigation entre les différents écrans grâce à un CardLayout.

Elle est responsable de :

- l'affichage du menu principal (choix du niveau),
- la transition vers l'écran d'amélioration des troupes,
- le lancement d'une partie,
- l'affichage de l'écran de fin,
- la gestion de la progression du joueur (via la classe Sauvegarde).

Elle instancie également les composants principaux du jeu :

- Partie (modèle),
- GameController (contrôleur),
- Affichage (vue).

Le démarrage du programme est effectué via :

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(Test::new);
}
```

8.2. Classes principales à connaître

Modèle

- **Partie.java** : contient l'état global du jeu (troupes, bâtiments, score, temps, étoiles, logique de combat).
- **Troupe.java** : classe abstraite représentant une unité (déplacement, attaque, points de vie).
- **Barbare / Sorcier / Pekka** : spécialisations de Troupe avec des caractéristiques différentes.
- **Batiment.java** : représente un bâtiment destructible.
- **Defense.java** : bâtiment offensif capable d'attaquer les troupes.
- **Chateau.java** : génère des troupes ennemies lorsqu'un joueur entre dans sa portée.
- **Ameliorations.java** : gère les niveaux des troupes et les coûts d'amélioration.
- **Sauvegarde.java** : gère l'or total du joueur et le déblocage des niveaux.

Vue

- **Affichage.java** : vue principale du jeu, responsable du rendu graphique
- **StartPanel.java** : écran de sélection des niveaux.
- **AmeliorationPanel.java** : écran permettant d'améliorer les troupes avant la partie.
- **FinPanel.java** : écran affiché à la fin d'une partie (score, étoiles, boutons).

Contrôleur

GameController.java : gère :
- les interactions utilisateur (clics),
- le déploiement des troupes,
- la sélection et le déplacement du Pekka,
- la boucle de jeu via deux threads :
  - threadJeu (40 ms) → mise à jour du jeu,
  - threadChrono (1 s) → gestion du temps.

Threads principaux

Le projet repose sur deux threads dans le contrôleur :

- **threadJeu** :

appelle partie.update(),
met à jour les déplacements, combats et score,
déclenche le repaint de la vue.

- **threadChrono** :

décrémente le temps restant,
déclenche la fin de partie si nécessaire.

8.3. Constantes importantes à modifier

Certaines constantes permettent de modifier facilement le comportement du jeu :

- **Dans les classes de troupes** :

points de vie (health),
dégâts (damage),
vitesse (speed).

- **Dans Defense.java** :

portée (portee),
cadence de tir.

- **Dans Ameliorations.java** :

coût des améliorations,
niveau maximum.

- **Dans GameController.java** :

vitesse du jeu (Thread.sleep(40)),
durée du chrono (Thread.sleep(1000)).

- **Dans la vue (Affichage.java)** :

dimensions de la grille,
styles graphiques.

8.4. Fonctionnalités pouvant être ajoutées

Plusieurs améliorations peuvent être envisagées :

- Ajout de nouveaux types de troupes avec des comportements spécifiques.
- Amélioration de l'intelligence des ennemis (priorités, stratégies).
- Ajout d'effets visuels (explosions, animations d'attaque).
- Système de pause / reprise de la partie ou arrêt de la partie.
- Ajout d'un système de sauvegarde plus avancé (progression détaillée).
- Ajout de nouveaux niveaux avec difficulté progressive.
- Amélioration de l'interface graphique (assets, animations, transitions).

9. Conclusion et perspectives

Ce projet s'inscrit dans la continuité du précédent mini-jeu et nous a permis d'aller plus loin dans la conception d'une application interactive en Java. Alors que le premier projet était centré sur un gameplay simple avec une animation continue, celui-ci introduit une structure plus complète avec plusieurs écrans, une logique de jeu plus riche et une véritable gestion d'état (partie, score, progression).

Il nous a permis de consolider nos acquis en Java, mais surtout de mieux comprendre comment organiser un projet de manière propre grâce à l'architecture MVC. On a également pu approfondir la gestion des interactions utilisateur.

Le résultat obtenu est plus abouti que le projet initial, avec un jeu structuré en plusieurs phases (menu, amélioration, combat, fin de partie) et une expérience utilisateur plus complète. Néanmoins, certaines améliorations pourraient encore être apportées, notamment en enrichissant le gameplay (nouvelles unités, comportements plus variés, intelligence des ennemis) ou en améliorant l'interface graphique.

Ce projet nous a également permis de mieux comprendre l'importance de la planification et de la documentation. Le fait de structurer le code et de réfléchir en amont à l'architecture peut aider à gagner du temps sur certaines parties, même si cela reste un point à améliorer pour nos futurs projets.

9.1. Retour sur les difficultés rencontrées

Plusieurs difficultés ont été rencontrées lors de ce projet.

Tout d'abord, la mise en place d'un système de jeu plus complexe a nécessité une bonne organisation des interactions entre les différentes classes, notamment pour gérer correctement les troupes, les bâtiments et le système de combat. Il a fallu plusieurs ajustements pour obtenir un comportement cohérent.

Ensuite, la transition vers une architecture MVC a été un point délicat. Contrairement au premier projet, où la séparation des responsabilités était très simple, il a été nécessaire ici de bien distinguer ce qui relève du modèle, de la vue et du contrôleur. Cette structuration a demandé un temps d'adaptation, notamment pour placer certaines classes ou gérer les interactions entre les composants.

La gestion des événements utilisateur a également été plus complexe, notamment pour interpréter correctement les clics (sélection de troupes, déploiement, interactions avec la carte) tout en conservant une séparation claire avec la logique métier.

Enfin, l'équilibrage du jeu et la gestion des différents niveaux ont nécessité plusieurs tests afin d'obtenir une progression cohérente et un gameplay intéressant.